

Code:

```
def evaluation_function(x):
    return -(x**2) + 5

def hill_climbing(start_x, step_size=0.1):
    current_x = round(start_x, 4)
    current_val = evaluation_function(current_x)

    step = 0
    print(f'Step {step}: Current state x = {current_x:.2f}, f(x) = {current_val:.4f}')

    while True:
        # Generate the two neighbouring states (x + step_size and x - step_size)
        neighbours = [round(current_x + step_size, 4), round(current_x - step_size, 4)]
        moved = False

        # Compare neighbours to current state
        for next_x in neighbours:
            next_val = evaluation_function(next_x)

            # If neighbour is better, move to it and continue search from new state
            if next_val > current_val:
                current_x = next_x
                current_val = next_val
                moved = True
                step += 1
                print(f'Step {step}: Moved to neighbour x = {current_x:.2f}, f(x) = {current_val:.4f}')
                break # Move immediately to this neighbour and search again

        # If all neighbours checked and none is better: stop
        if not moved:
            print("\nAll neighbours checked. None are better.")
            print("Stopping search: Reached peak / local maximum.")
            break

    return current_x, current_val

def main():
    start_input = float(input("Enter starting value of x: "))
    peak_x, peak_val = hill_climbing(start_input)
```

```
print("-" * 45)
print(f'Optimal x: {peak_x:.2f}')
print(f'Maximum f(x): {peak_val:.4f}')
```

```
if __name__ == "__main__":
    main()
```

Output:

```
Enter starting value of x: 2
Step 0: Current state x = 2.00, f(x) = 1.0000
Step 1: Moved to neighbour x = 1.90, f(x) = 1.3900
Step 2: Moved to neighbour x = 1.80, f(x) = 1.7600
Step 3: Moved to neighbour x = 1.70, f(x) = 2.1100
Step 4: Moved to neighbour x = 1.60, f(x) = 2.4400
Step 5: Moved to neighbour x = 1.50, f(x) = 2.7500
Step 6: Moved to neighbour x = 1.40, f(x) = 3.0400
Step 7: Moved to neighbour x = 1.30, f(x) = 3.3100
Step 8: Moved to neighbour x = 1.20, f(x) = 3.5600
Step 9: Moved to neighbour x = 1.10, f(x) = 3.7900
Step 10: Moved to neighbour x = 1.00, f(x) = 4.0000
Step 11: Moved to neighbour x = 0.90, f(x) = 4.1900
Step 12: Moved to neighbour x = 0.80, f(x) = 4.3600
Step 13: Moved to neighbour x = 0.70, f(x) = 4.5100
Step 14: Moved to neighbour x = 0.60, f(x) = 4.6400
Step 15: Moved to neighbour x = 0.50, f(x) = 4.7500
Step 16: Moved to neighbour x = 0.40, f(x) = 4.8400
Step 17: Moved to neighbour x = 0.30, f(x) = 4.9100
Step 18: Moved to neighbour x = 0.20, f(x) = 4.9600
Step 19: Moved to neighbour x = 0.10, f(x) = 4.9900
Step 20: Moved to neighbour x = 0.00, f(x) = 5.0000
```

All neighbours checked. None are better.
Stopping search: Reached peak / local maximum.

```
Optimal x: 0.00
Maximum f(x): 5.0000
```